

Methods and results: toward machine- ε strict- $h=0$ K=3 CRRA REE on the Chebyshev tabulated architecture

Fixed-Point Factory — auto-mode running notes

June 2026

Abstract

Cumulative document of methods explored to drive the K=3 CRRA REE fixed-point residual to machine ε on the tensor-Chebyshev tabulated- μ architecture. Two clean wins so far: **(1) kernel-band Cheb-tab nails to $\sim 10^{-16}$ from cold and warm** at bandwidths $h \in [0.20, 0.50]$; the FP depends on h and the joint $(N, h) \rightarrow (\infty, 0)$ limit gives slope $\alpha^* \approx 0.22$; **(2) partition-of-unity (POU) Cheb-tab at strict $h=0$ recovers the correct REE slope $\alpha^* \approx 0.37$ at $N=8$** , matching the legacy `hfree_smooth` at $G=9$. The POU operator floors at $\sim 4 \times 10^{-3}$ with generic Anderson+Newton-Krylov; several routes to lower (chebroots all-roots, analytic/complex-step Jacobian) are under investigation.

Contents

1	Background and where we started	1
2	Method 1: kernel-band co-area integral	1
3	Method 2: partition-of-unity (POU) at strict $h = 0$	2
3.1	Implementation: scan-and-bisect all-roots in numba	2
3.2	Results: correct strict- $h=0$ FP, floor $\sim 4 \times 10^{-3}$	2
4	Investigations into the 4×10^{-3} POU floor	3
4.1	Pure dense Newton with finite-difference Jacobian	3
4.2	Chebroots (companion-matrix all-roots) vs scan-and-bisect	3
5	Path forward to machine ε at strict $h = 0$	3

1 Background and where we started

The K=3 CRRA REE operator Φ takes a conjectured price function $P(u_1, u_2, u_3)$ and returns the next-iterate. The chebroots/bisect implementation we built initially floored at $\|F\|_\infty \sim 1.7 \times 10^{-3}$ across all solvers we tried (`fp_analysis.pdf`), diagnosed as structural from root-extraction non-smoothness. The tabulated- μ refactoring (`tab_explain.pdf`, $\sim 90\times$ speedup) was found to be *not* the cause of the floor: replaying a legacy operator that hit machine ε on its own grid and adding only the lookup table on top still reached 6.4×10^{-15} under Newton-Krylov.

2 Method 1: kernel-band co-area integral

Replace contour root extraction with a Gaussian band:

$$A_v(p, u_k) = \iint K_h(P(u_a, u_b, u_k) - p) f_v(u_a) f_v(u_b) du_a du_b, \quad K_h(z) = e^{-z^2/(2h^2)}.$$

With P as a tensor Chebyshev polynomial and the integral evaluated on a tensor-GL quadrature grid, A_v is an *analytic* function of the Cheb coefficients of P . Φ is then analytic in P , so Anderson and Newton–Krylov converge quadratically.

Result. Machine ε from both cold and warm starts at $h \in [0.20, 0.50]$, $G_p=121$, $N=6$ (Anderson floor $\sim 10^{-16}$, NK confirms to 10^{-15}). ~ 7 ms per Φ at $G_p=121$.

Caveat. The kernel-band FP *depends on h* . The joint ($N \rightarrow \infty, h \rightarrow 0$) limit (`kern.tab.pdf`) gives slope $\alpha^* \approx 0.22$ and deficit $1 - R^2 \approx 0.20$. Going below $h \approx 0.10$ at $N=12$ fails (the band tightens past the grid resolution and the operator approaches the discontinuous strict- $h=0$ limit).

3 Method 2: partition-of-unity (POU) at strict $h = 0$

NO kernel, NO bandwidth. The strict co-area integral $A_v = \int_{P=p} f_v f_v / |\nabla P| d\sigma$ is made smooth by representing it as the sum of two parameterizations:

$$A_v = A_v^{(a)} + A_v^{(b)},$$

where $A_v^{(b)}$ parameterizes the contour by u_a (root in u_b at each GL node) and $A_v^{(a)}$ by u_b (root in u_a). Each term is weighted by a partition-of-unity factor $w_a = d_a^2 / (d_a^2 + d_b^2)$, $w_b = d_b^2 / (d_a^2 + d_b^2)$, with $w_a + w_b = 1$. As $|d_b| \rightarrow 0$ (turning point along u_b), $w_b \rightarrow 0$ and the $1/|d_b|$ singularity is killed; the contribution shifts to $A^{(a)}$.

This is a direct port of the K=3 `hfree_smooth` legacy operator (`projects/REZN/solved_fixed_points/k3_hfree`) to the Cheb-tab architecture. The atanh -stretched u -coordinate makes $f_v(u(\xi = \pm 1)) = 0$ exactly, so roots crossing the box edge contribute zero amplitude smoothly — no bandwidth or cutoff needed for the boundary behaviour.

3.1 Implementation: scan-and-bisect all-roots in numba

All-roots-in- $[-1, 1]$ via dense sign-change scan ($8N$ subintervals) + 80-iter bisection, fully numba-jit'd: `cheby_numba_pou_tab.py`.

Per- Φ wall time at $G_p=121$:

N	G	ms / Φ
6	7	52
8	9	93
10	11	188

3.2 Results: correct strict- $h=0$ FP, floor $\sim 4 \times 10^{-3}$

N (G)	best $\ F\ _\infty$	slope α^*	deficit
6 ($G=7$)	5.7×10^{-3}	0.336	0.139
8 ($G=9$)	3.9×10^{-3}	0.366	0.177
10 ($G=11$)	1.3×10^{-2}	0.439	0.165 (cold-start basin issue)
Legacy hfree_smooth ($G=9$, spline+exact NK)	4×10^{-5}	0.364	0.173

The POU $N=8$ slope and deficit agree with the legacy to three digits — the same strict- $h=0$ REE. Our generic Anderson+NK floors $\sim 100\times$ above the legacy because the legacy used exact-Jacobian Newton steps that contract quadratically.

4 Investigations into the 4×10^{-3} POU floor

4.1 Pure dense Newton with finite-difference Jacobian

40-DOF system. FD Jacobian costs $\sim 40 \cdot \Phi = 2$ s per step. Sweep $\varepsilon_{\text{FD}} \in \{10^{-5}, 10^{-6}, 10^{-7}, 10^{-8}\}$, warm-start from POU Anderson best:

ε_{FD}	best $\ F\ _\infty$
10^{-5}	3.70×10^{-3}
10^{-6}	2.71×10^{-3}
10^{-7}	3.79×10^{-3}
10^{-8}	3.82×10^{-3}

Marginal improvement over Anderson (4.1e-3). Line-search alpha collapses to 10^{-9} , confirming the Jacobian carries non-smoothness noise: **the operator, not the solver, is the bottleneck.**

4.2 Chebroots (companion-matrix all-roots) vs scan-and-bisect

Hypothesis: my numba scan-and-bisect misses roots near the box boundary, contributing to the floor. Test: replace with `numpy.polynomial.chebyshev.chebroots` (companion matrix eigenvalues, finds all real roots exactly):

all-roots method	ms / Φ	max $ \Delta P $ vs scan-bisect
scan-and-bisect (numba)	50	— (reference)
chebroots (numpy)	2200	3.7×10^{-4}

The per- Φ difference is the same order as our floor. Strong evidence that scan-and-bisect misses roots near the boundary and this is what limits POU convergence. The natural fix is to port the chebroots-based all-roots to numba (companion matrix eigenvalues are well-supported in numba's `linalg.eigvals`) or to dramatically increase the scan resolution near the boundary.

5 Path forward to machine ε at strict $h = 0$

If the chebroots experiment confirms much lower POU floor, the next implementation step is straightforward:

1. Port chebroots (companion matrix) to numba — Cheb’s comrade matrix at $N = 6$ is 6×6 , eigenvalue extraction is fast and supported in numba.
2. Re-run POU Anderson+NK; expect floor $\rightarrow 10^{-7}$ or lower if root-finding completeness was the bottleneck.

If FD-Newton still floors above 10^{-7} after fixing the root-finding, the remaining route is **analytic-Jacobian Newton** (the legacy path):

1. Implement complex-step differentiation: replace all float64 in the POU operator with complex128, compute $\partial F / \partial x_j = \text{Im}[F(x + i h e_j)] / h$ with $h = 10^{-30}$. Exact to machine ε (no FD cancellation).
2. Or: derive $\partial A_v / \partial P_{\text{coeffs}}$ in closed form. POU is differentiable (Newton-tracked roots give smooth $\partial \xi_b^* / \partial P_{\text{coeffs}} = -\partial_{\text{coeff}} P / \partial \xi_b P$ at the root); the per- P -coeff sensitivity of A_v chains through the partition weights, Bayes update, and CRRA clearing.

The kernel-band Cheb-tab *already* reaches machine ε for the smoothed operator. The remaining question is whether the *strict* $h=0$ FP (which is the actual REE) is reachable at machine ε in pure float64 without arb-precision arithmetic. The legacy *k3_hfree_100* work shows it can be nailed to 100 decimals in arb; whether float64 is enough is an empirical question.